

EIRC Technology Overview

The Eclipse technology consists of resource preparation, image processing research focusing on facial detection, implementation of a face processing engine, and enrollment management. In this document, intellectual property will be marked by  **IP** , and unique implementations of prior art will be marked  **Key** .

The Eclipse Face Processing (EFP) Engine includes the following steps:

- | | |
|-----------------------------------|---------------------------|
| 1. Image Acquisition | 5. Face Vector Generation |
| 2. Frontal Face Detection | 6. Casual Matching |
| 3. Facial Feature (eyes) Location | 7. Image Output |
| 4. Face Image Normalization | 8. Face-based Analytics |

EFP Engine

Image Acquisition

The engine acquires frame images from either a HTTP-based IP camera, such as the Axis series or a directory visible to the local machine. The 'hot-directory' concept  **Key** will wait for files to be dropped into an empty directory. A loop flag allows the directory to be continuously processed for demonstration purposes.

During the acquisition process, frame images are converted to a consistent RGBA32 format for output images and 8-bit greyscale for object detection purposes. Images can be scaled, typically from 4x to 0.25x to optimized the detection process. Facial recognition is optimized between 64 and 256 pixels between the eyes. But usable results are possible with 16, so the 4x scale can be helpful in those conditions. As well as quarter (or less) scale for multi-megapixel headshots where the extra pixels just waste time and memory.

Frontal Face Detection

Face detection uses OpenCV's Haar Cascade processing to return a list of rectangles where potential faces are detected. The process is roughly as follows:

- 1) Load a trained Cascade Classifier. (See Face Resources below.)
- 2) Convert a greyscale frame to OpenCV image matrix format.
- 3) Prepare parameters for detection arguments.
 - a) Scale Factor, calculated from Min/MaxAcross, Min/MaxPixels, MaxDensity, and MaxDetectors, overridden by Factor .
 - b) Minimum Neighbors, calculated from MinQuality.  **IP**
 - c) Minimum and Maximum Size calculated from Min/MaxAcross  **IP** overridden by Min/MaxPixels .
 - d) Grouping detections using one of three methods and their thresholds. Assigning a face detect quality score to each grouped set of rectangles.  **IP** *Note: Newer OpenCV implementations do the grouping, but our assigning quality levels is unique in the industry; others just treat it like a light switch.*

Haar Cascades

The object detector described below has been initially proposed by Paul Viola [289] and improved by Rainer Lienhart [171] .

First, a classifier (namely a cascade of boosted classifiers working with haar-like features) is trained with a few hundred sample views of a particular object (i.e., a face or a car), called positive examples, that are scaled to the same size (say, 20x20), and negative examples - arbitrary images of the same size.

After a classifier is trained, it can be applied to a region of interest (of the same size as used during the training) in an input image. The classifier outputs a "1" if the region is likely to show the object (i.e., face/car), and "0" otherwise. To search for the object in the whole image one can move the search window across the image and check every location using the classifier. The classifier is designed so that it can be easily "resized" in order to be able to find the objects of interest at different sizes, which is more efficient than resizing the image itself. So, to find an object of an unknown size in the image the scan procedure should be done several times at different scales.

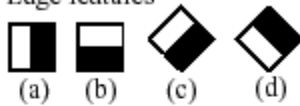
The word "cascade" in the classifier name means that the resultant classifier consists of several simpler classifiers (stages) that are applied subsequently to a region of interest until at some stage the candidate is rejected or all the stages are passed. The word "boosted" means that the classifiers at every stage of the cascade are complex themselves and they are built out of basic classifiers using one of four different boosting techniques (weighted voting). Currently Discrete Adaboost, Real Adaboost, Gentle Adaboost and Logitboost are supported. The basic classifiers are decision-tree classifiers with at least 2 leaves. Haar-like features are the input to the basic classifiers, and are calculated as described below. The current algorithm uses the following Haar-like features:

The feature used in a particular classifier is specified by its shape (1a, 2b etc.), position within the region of interest and the scale (this scale is not the same as the scale used at the detection stage, though these two scales are multiplied). For example, in the case of the third line feature (2c) the response is calculated as the difference between the sum of image pixels under the rectangle covering the whole feature (including the two white stripes and the black stripe in the middle) and the sum of the image pixels under the black stripe multiplied by 3 in order to compensate for the differences in the size of areas. The sums of pixel values over a rectangular regions are calculated rapidly using integral images (see below and the integral description).

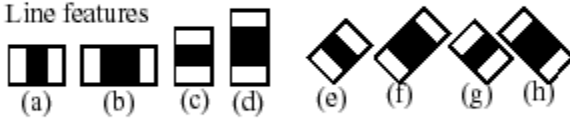
Check [the corresponding tutorial](#) for more details.

The following reference is for the detection part only. There is a separate application called `opencv_traincascade` that can train a cascade of boosted classifiers from a set of samples.

1. Edge features



2. Line features



3. Center-surround features

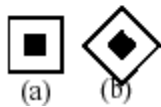


Figure 1: Haar-like Features

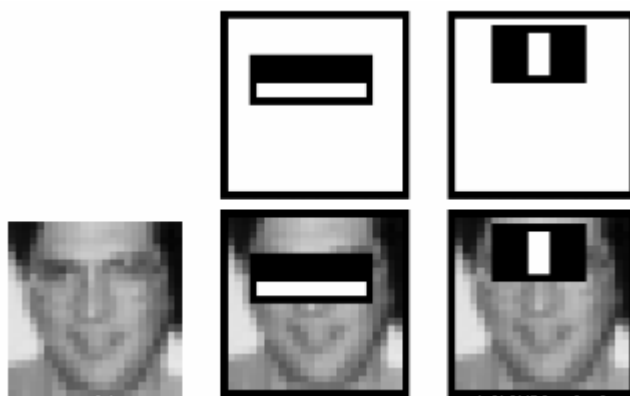


Figure 2: Haar Face

Haar Tutorial

Goal

In this tutorial,

We will learn how the Haar cascade object detection works.

We will see the basics of face detection and eye detection using the Haar Feature-based Cascade Classifiers

We will use the `cv::CascadeClassifier` class to detect objects in a video stream. Particularly, we will use the functions:

`cv::CascadeClassifier::load` to load a .xml classifier file. It can be either a Haar or a LBP classifier

`cv::CascadeClassifier::detectMultiScale` to perform the detection.

Theory

Object Detection using Haar feature-based cascade classifiers is an effective object detection method proposed by Paul Viola and Michael Jones in their paper, "Rapid Object Detection using a Boosted Cascade of Simple Features" in 2001. It is a machine learning based approach where a cascade function is trained from a lot of positive and negative images. It is then used to detect objects in other images.

Here we will work with face detection. Initially, the algorithm needs a lot of positive images (images of faces) and negative images (images without faces) to train the classifier. Then we need to extract features from it. For this, Haar features shown in the below image are used. They are just like our convolutional kernel. Each feature is a single value obtained by subtracting sum of pixels under the white rectangle from sum of pixels under the black rectangle.

Now, all possible sizes and locations of each kernel are used to calculate lots of features. (Just imagine how much computation it needs? Even a 24x24 window results over 160000 features). For each feature calculation, we need to find the sum of the pixels under white and black rectangles. To solve this, they introduced the integral image. However large your image, it reduces the calculations for a given pixel to an operation involving just four pixels. Nice, isn't it? It makes things super-fast.

But among all these features we calculated, most of them are irrelevant. For example, consider the image below. The top row shows two good features. The first feature selected seems to focus on the property that the region of the eyes is often darker than the region of the nose and cheeks. The second feature selected relies on the property that the eyes are

darker than the bridge of the nose. But the same windows applied to cheeks or any other place is irrelevant. So how do we select the best features out of 160000+ features? It is achieved by Adaboost.

For this, we apply each and every feature on all the training images. For each feature, it finds the best threshold which will classify the faces to positive and negative. Obviously, there will be errors or misclassifications. We select the features with minimum error rate, which means they are the features that most accurately classify the face and non-face images. (The process is not as simple as this. Each image is given an equal weight in the beginning. After each classification, weights of misclassified images are increased. Then the same process is done. New error rates are calculated. Also new weights. The process is continued until the required accuracy or error rate is achieved or the required number of features are found).

The final classifier is a weighted sum of these weak classifiers. It is called weak because it alone can't classify the image, but together with others forms a strong classifier. The paper says even 200 features provide detection with 95% accuracy. Their final setup had around 6000 features. (Imagine a reduction from 160000+ features to 6000 features. That is a big gain).

So now you take an image. Take each 24x24 window. Apply 6000 features to it. Check if it is face or not. Wow.. Isn't it a little inefficient and time consuming? Yes, it is. The authors have a good solution for that.

In an image, most of the image is non-face region. So it is a better idea to have a simple method to check if a window is not a face region. If it is not, discard it in a single shot, and don't process it again. Instead, focus on regions where there can be a face. This way, we spend more time checking possible face regions.

For this they introduced the concept of Cascade of Classifiers. Instead of applying all 6000 features on a window, the features are grouped into different stages of classifiers and applied one-by-one. (Normally the first few stages will contain very many fewer features). If a window fails the first stage, discard it. We don't consider the remaining features on it. If it passes, apply the second stage of features and continue the process. The window which passes all stages is a face region. How is that plan!

The authors' detector had 6000+ features with 38 stages with 1, 10, 25, 25 and 50 features in the first five stages. (The two features in the above image are actually obtained as the best two features from Adaboost). According to the authors, on average 10 features out of 6000+ are evaluated per sub-window.

Haar-cascade Detection in OpenCV

C++

OpenCV provides a training method (see [Cascade Classifier Training](#)) or pretrained models, that can be read using the `cv::CascadeClassifier::load` method. The pretrained models are located in the data folder in the OpenCV installation or can be found [here](#).

First, a `cv::CascadeClassifier` is created and the necessary XML file is loaded using the `cv::CascadeClassifier::load` method. Afterwards, the detection is done using the `cv::CascadeClassifier::detectMultiScale` method, which returns boundary rectangles for the detected faces or eyes.

Facial Feature Location

Face Image Normalization

Face Vector Generation

Casual Matching

Image Output

Face-based Analytics

C³: Configuration, Command, Control

Resources

Face Cascade Training

Vector Generation

Enrollment

Person Mode

Research